



PyQuest

Information- & Aufgabenblatt

Kapitel 9: Listen

Speichere mehrere Werte in einer einzigen Variable.

Listen erstellen und lesen

Eine **Liste** speichert mehrere Werte in einer einzigen Variable, in eckigen Klammern [], getrennt durch Kommas.

Auf ein Element zugreifst du über seinen **Index** – und der beginnt bei **0**, nicht bei 1!

Beispiel

```
farben = ["rot", "gruen", "blau"]  
print(farben[0])  
print(farben[2])
```

farben[0] ist das **erste** Element ("rot"), farben[2] das **dritte** ("blau"). Mit farben[-1] bekommst du das **letzte** Element, egal wie lang die Liste ist. Mit len(farben) erfährst du, wie viele Elemente die Liste hat.

Aufgabe 1: Welchen Index hat das erste Element einer Liste?

- ☐ a) 1
- ☐ b) 0
- ☐ c) -1

Aufgabe 2: Lege eine Liste **tiere** mit den Werten "Hund", "Katze", "Maus" an und gib das zweite Element aus (soll Katze sein).

Dein Code:



Listen verändern

Listen kannst du nachträglich verändern:

- `.append(wert)` fügt ein neues Element **am Ende** hinzu
- `.remove(wert)` entfernt das erste Vorkommen eines Werts
- `liste[i] = neuer_wert` ersetzt das Element an Position `i`

Ein Element hinzufügen:

Beispiel

```
einkaufsliste = ["Brot", "Milch"]  
einkaufsliste.append("Eier")  
print(einkaufsliste)
```

Beim Ausgeben einer ganzen Liste zeigt Python sie in eckigen Klammern mit allen Elementen an – genau wie du sie angelegt hast.

Aufgabe 3: Was macht `.append(wert)` bei einer Liste?

- ☐ a) Fügt den Wert am Ende der Liste hinzu
- ☐ b) Ersetzt das erste Element
- ☐ c) Löscht die Liste

Aufgabe 4: Lege eine leere Liste namen an. Füge nacheinander mit `.append()` die Werte "Ada", "Alan" und "Grace" hinzu. Gib die Liste am Ende aus.

Dein Code:



Listen mit while durchlaufen

Oft willst du **jedes Element** einer Liste bearbeiten. Mit den Bausteinen, die du schon kennst, geht das so: Du nimmst eine while-Schleife und eine **Index-Variable** *i*, die bei 0 startet.

Mit `liste[i]` holst du das Element an Position *i*, und mit `len(liste)` weißt du, wie viele Elemente es gibt. Die Schleife läuft, **solange** *i* kleiner als die Länge der Liste ist.

Jede Farbe der Reihe nach ausgeben – *i* läuft von 0 bis zum letzten gültigen Index:

Beispiel

```
farben = ["rot", "gruen", "blau"]
i = 0
while i < len(farben):
    print(farben[i])
    i += 1
```

Wichtig ist wieder das **Hochzählen** mit `i += 1` – sonst bleibt *i* bei 0 stehen und du hast eine Endlosschleife. Die Bedingung `i < len(farben)` sorgt dafür, dass die Schleife **genau so oft** läuft, wie die Liste Elemente hat (der letzte gültige Index ist immer `len(...) - 1`).

Aufgabe 5: Welche Bedingung sorgt dafür, dass die while-Schleife genau alle Elemente der Liste *farben* durchläuft?

- ☐ a) `while i < len(farben):`
- ☐ b) `while i <= len(farben):`
- ☐ c) `while farben > 0:`

Aufgabe 6: Lege die Liste `zahlen = [3, 7, 2, 9, 4]` an. Durchlaufe sie mit einer while-Schleife und einer Index-Variable und gib nur die Zahlen aus, die größer als 4 sind.

Erwartet: 7, dann 9 (untereinander).

Dein Code:



Das funktioniert – ist aber ganz schön umständlich mit `i`, `len()` und Hochzählen. **Merke dir das Beispiel:** Im nächsten Kapitel lernst du die **for-Schleife** kennen, mit der genau dasselbe in **einer** Zeile ohne Index gelingt. 😊

Listen verbinden und Slicing

Mit `+` kannst du zwei Listen zu einer **neuen** Liste **verbinden** (verketteten) – genau wie du es von Strings kennst.

`+` hängt `liste2` hinten an `liste1` an:

Beispiel

```
liste1 = [1, 2, 3]
liste2 = [4, 5, 6]
kombiniert = liste1 + liste2
print(kombiniert)
```

Auch **Slicing** funktioniert bei Listen genauso wie bei Strings: `liste[start:ende]` gibt eine **neue Teilliste** zurück – der `ende`-Index ist wieder **ausgeschlossen**.

`liste[1:4]` holt die Elemente an den Indizes 1, 2 und 3:

Beispiel

```
zahlen = [10, 20, 30, 40, 50]
teil = zahlen[1:4]
print(teil)
```

Aufgabe 7: Was gibt `[1, 2, 3] + [4, 5]` zurück?

- ☐ a) `[5, 7]`
- ☐ b) `[1, 2, 3, 4, 5]`
- ☐ c) Einen Fehler



Aufgabe 8: In `gruppe_a` und `gruppe_b` stehen zwei Listen. Verbinde sie mit `+` zu `alle`, gib `alle` aus. Gib danach mit Slicing nur die letzten zwei Namen von `alle` aus (`alle[-2:]`).

```
gruppe_a = ["Ada", "Alan"]  
gruppe_b = ["Grace", "Linus", "Tim"]
```

Dein Code:

Suchen und Entfernen

Du kennst `in` schon von Strings – bei Listen funktioniert es genauso: Es prüft, ob ein Wert **enthalten** ist, und liefert `True` oder `False`.

Mit `.index(wert)` findest du die **Position** eines Werts (den Index des **ersten** Treffers) – ähnlich wie `.find()` bei Strings. Anders als `.find()` wirft `.index()` aber einen **Fehler**, wenn der Wert nicht existiert.

`in` prüft die Existenz, `index()` liefert die Position:

Beispiel

```
tiere = ["Hund", "Katze", "Maus"]  
print("Katze" in tiere)  
print(tiere.index("Katze"))
```

Mit `.count(wert)` zählst du, **wie oft** ein Wert in der Liste vorkommt. Und mit `.remove(wert)` entfernst du das **erste** Vorkommen eines Werts aus der Liste (die Liste wird dabei direkt verändert).

`count()` zählt Vorkommen, `remove()` löscht das erste Vorkommen:

Beispiel

```
zahlen = [3, 7, 3, 9, 3]  
print(zahlen.count(3))  
zahlen.remove(3)  
print(zahlen)
```

**Aufgabe 9: Was macht `.remove(wert)`, wenn der Wert mehrfach in der Liste vorkommt?**

- ☐ a) Entfernt alle Vorkommen
- ☐ b) Entfernt nur das erste Vorkommen
- ☐ c) Gibt einen Fehler zurück

Aufgabe 10: In `noten` steht eine Liste mit Schulnoten. Gib aus, wie oft die Note 3 vorkommt (`.count()`). Entferne danach eine 5 mit `.remove()` und gib die komplette Liste aus.

```
noten = [2, 3, 5, 1, 3, 5, 4]
```

Dein Code:

Listen anwenden

Jetzt kombinierst du **Listen** mit der **while-Schleife** – genau wie in der letzten Lektion des Schleifen-Kapitels: eine Index-Variable `i`, die Bedingung `i < len(liste)`, und Zugriff über `liste[i]`.

Aufgabe: In `zahlen` fehlt **genau eine** Zahl aus der Reihe 1 bis 10. Finde sie mit einem mathematischen Trick: Die **Soll-Summe** von 1 bis 10 lässt sich direkt berechnen ($n * (n + 1) // 2$, mit $n = 10$). Die **Ist-Summe** bildest du selbst über die Liste. Die **Differenz** ist die fehlende Zahl!

Soll-Summe direkt berechnet, Ist-Summe über die Liste aufsummiert:

Beispiel

```
n = 5
soll_summe = n * (n + 1) // 2
print(soll_summe)
```



Aufgabe 11: In `zahlen` fehlt eine Zahl aus der Reihe 1 bis 10. Berechne: 1. `soll_summe` – die Summe von 1 bis 10, direkt mit der Formel $n * (n + 1) // 2$ ($n = 10$) 2. `ist_summe` – die tatsächliche Summe der Liste, mit einer `while`-Schleife und `Index` aufsummiert 3.

`fehlende_zahl` – die Differenz `soll_summe - ist_summe`

Gib am Ende nur `fehlende_zahl` aus.

```
zahlen = [1, 2, 4, 5, 6, 7, 8, 9, 10]
```

Dein Code:

Ein weiterer typischer Einsatz: **prüfen, ob es Duplikate gibt**. Dafür baust du eine **zweite, leere Liste** auf – die „schon gesehen“-Liste. Für jedes Element prüfst du mit `in`, ob es dort **schon drin** ist. Wenn ja, hast du ein Duplikat gefunden!

Aufgabe 12: In `zahlen` steht eine Liste mit genau einer doppelten Zahl. Finde sie: Lege eine leere Liste `gesehen = []` an. Durchlaufe `zahlen` mit `while` und `Index`. Ist das aktuelle Element bereits in `gesehen`, gib es aus und beende die Schleife mit `break`. Sonst füge es mit `.append()` zu `gesehen` hinzu.

```
zahlen = [4, 8, 15, 16, 23, 8, 42]
```

Dein Code: